

1. Generic Environment Setup

We recommend installing all dependencies in a fresh PIP environment. We will configure a generic PIP environment to manage your PyFlow-specific Python runtime and dependencies.

1.1 MacRTL workstations and personal machines

Check if you already have Python installed by running the following in a terminal:

```
which python3
```

If this returns a valid path (e.g., `/usr/bin/python3`), then proceed to Step 2. Otherwise, continue with Step 1.1.1.

1.1.1 Installing Python

We will not install the latest version of Python (currently 3.13.7), as not all packages (e.g., PyTorch) are fully compatible with it. Instead, we will **install the latest version of Python 3.12**.

Linux users: Start with the system Python version. If it is not installed, your package manager will have a recent version. For example, on Ubuntu, you would use

```
sudo apt install python3-dev
```

Windows users: I highly recommend using Windows Subsystem for Linux (WSL). Its package manager will function almost identically to that of a standard Linux distribution.

Mac users: MacOS does not have a package manager, so you will need to follow these instructions.

1. Navigate to <https://www.python.org/downloads/>.
2. Under "Looking for Python with a different OS?", click on the link to MacOS.
3. Find the latest version of Python 3.12 (e.g., 3.12.9) and download the installer.
4. Open the installer and follow the instructions.

1.2 Notre Dame CRC cluster

We have pre-built CPU and GPU environments for CRC clusters. See the [instructions for activating the shared PyFlow environments](#).

1.3 Supercomputers

These likely already have a version of PyTorch that you should use. You still might need to install dependencies. See [supercomputing](#) for site-specific instructions.

2. Create a PIP Virtual Environment and Install PyTorch

Because we will build some dependencies from source, it is best to start with a fresh PIP environment dedicated to PyFlow. Create a PyFlow-specific environment in your terminal:

```
python3 -m venv ~/venv/pyflow
```

Activate this environment:

```
source ~/venv/pyflow/bin/activate
```

Next, install the latest versions of PyTorch and Numpy. The PyTorch installer should autodetect an NVIDIA GPU if your machine has one.

```
pip3 install torch  
pip3 install numpy
```

3. Installing Dependencies

We will build the following PyFlowCL dependencies from source:

- An MPI implementation (if necessary)
- The HDF5 library
- mpi4py
- h5py
- HYPRE

Building HDF5 from source is necessary to enable parallel file I/O (it is not included in the versions of h5py available from Conda, PIP, etc).

3.0.1 Instructions for Clusters

NOTE: For the Notre Dame CRC cluster, see the [instructions for activating the group Conda environment](#).

All clusters will have at least one compiler and MPI implementation installed. The `module avail` command will list these. Often, the number of options will be overwhelming. The "**Golden Rule**" for compiled libraries is:

The compiler version you load must match the compiler version that was used to compile MPI.

Most clusters handle this automatically. Some general rules are:

1. AMD CPUs: use the latest version of GCC

2. Intel CPUs: use the latest version of the Intel compilers
3. Cray machines: use the default Cray compilers (regardless of CPU manufacturer)

3.0.2 Instructions for MacRTL Workstations & Personal Machines

- **MacRTL workstations and Linux-based personal machines:** you will use a recent (>6.0) version of GCC. To determine the version of GCC that is loaded in your terminal, use `gcc --version`. You may install a more recent version using your package manager, e.g., `dnf` (Fedora/Red Hat), `apt-get` (Ubuntu), etc.
- **MacOS:** the default version is not actually GCC at all but rather a version of LLVM Clang. Because it guarantees maximum compatibility, all precompiled Python libraries for MacOS (i.e., those available from Conda, PIP, etc.) are compiled using Clang. Therefore, for compatibility, we need to compile all PyFlowCL dependencies (OpenMPI, HDF5, mpi4py, and h5py) using the *system default version of Clang*.

Our workstations and most personal machines do not come with an MPI implementation installed. You will need to install your own; for this, follow the instructions for OpenMPI Installation in Section 3.2.

3.1 Setting the Install Directory

We will set up an install directory within your home directory called `LIB_DIR`. Check the output of `gcc --version` (if using GCC) and label the directory with your compiler's version number. For example:

- **Linux**, GCC 11.3.0 (e.g. workstations), where we are going to install OpenMPI 4.1.4:

```
export LIB_DIR=$HOME/support-gcc-11.3.0_openmpi-4.1.4
```

- On **MacOS**, my GCC is actually Clang-11.0.3, so I would do:

```
export LIB_DIR=$HOME/support-clang-11.0.3_openmpi-4.1.4
```

After setting the `LIB_DIR` shell variable, create the directory and subdirectories, and move into `$LIB_DIR/Build`:

```
cd
mkdir -p $LIB_DIR && cd $LIB_DIR
mkdir -p Build Install && cd Build
```

Open your `~/ .bashrc` file and add the following three lines, replacing `<lib_dir name>` with your chosen folder name.

```
export LIB_DIR=$HOME/<lib_dir name>
export PATH=$LIB_DIR/Install/bin:$PATH
export LD_LIBRARY_PATH=$LIB_DIR/Install/lib:$LD_LIBRARY_PATH
```

Finally, reload `.bashrc` to capture these changes to `PATH` and `LD_LIBRARY_PATH`:

```
source ~/.bashrc
```

3.2 OpenMPI Installation

Note: Skip this section if your machine already has an MPI implementation installed. This is true for virtually every computing cluster. You **will** need to build MPI for MacRTL workstations. To check if MPI is installed, use the following commands:

```
which mpicc
mpicc --version
```

The Message Passing Interface (MPI) is a standard for libraries that allows code parallelization on distributed memory systems. An MPI implementation is required to compile HDF5, mpi4py, and h5py.

[OpenMPI](#) is the recommended distribution; installation instructions follow.

Download and unpack the OpenMPI archive (here, version 4.1.4) in `$LIB_DIR/Build`.

```
wget https://download.open-mpi.org/release/open-mpi/v4.1/openmpi-4.1.4.tar.gz
tar xvf openmpi-4.1.4.tar.gz
cd openmpi-4.1.4
```

To automatically generate a *Makefile* for OpenMPI, run the *configure* script as follows. The option `prefix` specifies where the program will be installed.

- **For GCC**, execute

```
./configure CC=gcc CXX=g++ CFLAGS="-O3 -march=native" CXXFLAGS="-O3 -march=native" \
  --prefix=$LIB_DIR/Install
```

- **For Intel compilers**, use

```
./configure CC=icc CXX=icpc CFLAGS="-O3 -xhost -ip" CXXFLAGS="-O3 -xhost -ip" \  
    --prefix=$LIB_DIR/Install
```

Compile and install the library. (The `-j 4` specifies parallelization using four cores. If you are compiling on a workstation, you can use all cores by running `make -j` [no numerical argument].)

```
make -j 8  
make install
```

Upon successful installation, you should find the binaries *mpicc* and *mpicxx* in `$LIB_DIR/Install/bin`:

```
ls -l $LIB_DIR/Install/bin
```

3.3 HDF5 Installation

We need to compile the HDF5 backend library (C/C++) before we can install h5py (Python). HDF5 needs to be compiled from source in order to use the system's MPI implementation.

Building HDF5 requires CMake. On the **MacRTL workstations**, run

```
module load cmake
```

On personal machines, you will have to install CMake using your default package manager.

Currently (03/03/2023), v1.12.2 is working for PyFlowCL. Be aware that even version numbers (i.e. 1.12.x) are stable while odd version numbers (i.e. 1.13.x) are unstable, so we will stick to using even versions.

Download HDF5 to `$LIB_DIR/Build` and unzip its contents.

```
cd $LIB_DIR/Build  
wget https://support.hdfgroup.org/ftp/HDF5/releases/hdf5-1.12/hdf5-  
1.12.2/src/hdf5-1.12.2.tar.gz  
tar xf hdf5-1.12.2.tar.gz  
cd hdf5-1.12.2  
mkdir build && cd build
```

Next, export compiler optimization flags as environment variables.

- **For GCC compilers** on generic x86 architectures (AMD or Intel), use:

```
export CFLAGS="-O3 -march=native"
export CXXFLAGS="-O3 -march=native"
export FCFLAGS="-O3 -march=native"
```

- **For Intel compilers** on Intel CPUs, use:

```
export CFLAGS="-O3 -xhost -ip"
export CXXFLAGS="-O3 -xhost -ip"
export FCFLAGS="-O3 -xhost -ip"
```

Run CMake to configure the build:

```
CC=mpicc CXX=mpicxx cmake ../ -DHDF5_ENABLE_PARALLEL=YES -
DCMAKE_INSTALL_PREFIX=$LIB_DIR/Install
```

If CMake returns successfully, then compile the code. You may speed up compilation using, e.g., `make -j 4` (to compile using 4 cores) or `make -j` (to compile using all cores).

```
make -j 8
make install
```

Upon successful installation, the library `libhdf5.a` should appear in `$LIB_DIR/Install/lib`.

3.4 h5py Installation

We need to build a version of h5py that links to the previously installed HDF5 library. *This will also build mpi4py*. Ensure that your PIP environment is loaded, e.g., `source ~/venv/pyflow/bin/activate`.

Next, compile h5py using PIP (note that this requires `$LIB_DIR` to be set correctly):

```
HDF5_DIR=$LIB_DIR/Install
LD_LIBRARY_PATH=$LIB_DIR/Install/lib:$LD_LIBRARY_PATH \
    HDF5_MPI="ON" CC=mpicc CFLAGS="-I$LIB_DIR/Install/include -
L$LIB_DIR/Install/lib" \
    pip install --no-binary=h5py h5py
```

If successful, then you should see a message "Successfully installed h5py."

This step should also install mpi4py as a dependency. Check the PIP output to verify that mpi4py was installed.

3.5 HYPRE Installation

Ensure that your PIP environment is loaded, e.g., `source ~/venv/pyflow/bin/activate`.

Clone the HYPRE development branch:

```
cd $LIB_DIR/Build
git clone https://github.com/hypre-space/hypre.git hypre
cd hypre/src
mkdir build && cd build
```

Check out the latest release branch -- check [here](#) for latest release:

```
git checkout v3.2.0
```

CPU Builds

Configure build using CMake:

```
cmake .. \
  -DCMAKE_INSTALL_PREFIX=$LIB_DIR/Install \
  -DCMAKE_C_COMPILER=mpicc \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DCMAKE_Fortran_COMPILER=mpif90 \
  -DCMAKE_C_FLAGS='-fPIC -fstack-protector -g3 -O3' \
  -DCMAKE_CXX_FLAGS='-fstack-protector -g3 -O3 -fPIC' \
  -DCMAKE_Fortran_FLAGS='-fPIC -ffree-line-length-0 -g3 -O3' \
  -DBUILD_SHARED_LIBS=ON \
  -DTPL_BLAS_LIBRARIES=OFF \
  -DTPL_LAPACK_LIBRARIES=OFF \
  -DHYPRE_ENABLE_MPI=ON \
  -DHYPRE_ENABLE_OPENMP=ON \
  -DHYPRE_ENABLE_FORTRAN=ON
```

Build:

```
make -j
make install
```

GPU Builds (CUDA only)

Configure build using CMake:

```
cmake .. \  
  -DCMAKE_INSTALL_PREFIX=$LIB_DIR/Install \  
  -DCMAKE_C_COMPILER=mpicc \  
  -DCMAKE_CXX_COMPILER=mpicxx \  
  -DCMAKE_Fortran_COMPILER=mpif90 \  
  -DCMAKE_C_FLAGS='-fPIC -fstack-protector -g3 -O3' \  
  -DCMAKE_CXX_FLAGS='-fstack-protector -g3 -O3 -fPIC' \  
  -DCMAKE_Fortran_FLAGS='-fPIC -ffree-line-length-0 -g3 -O3' \  
  -DBUILD_SHARED_LIBS=ON \  
  -DTPL_BLAS_LIBRARIES=OFF \  
  -DTPL_LAPACK_LIBRARIES=OFF \  
  -DHYPRE_ENABLE_CUDA=ON \  
  -DHYPRE_ENABLE_UMPIRE=ON \  
  -DHYPRE_BUILD_UMPIRE=ON \  
  -DHYPRE_ENABLE_OPENMP=ON \  
  -DHYPRE_ENABLE_MPI=ON
```

Build:

```
make -j  
make install
```

3.6 PyFlowFV Additional Libraries

[Building additional PyFlowFV libraries](#)

3.7 Additional Libraries

You'll probably want to install some additional Python libraries in your environment. If necessary, activate your Pip environment ([source ~/venv/pyflow/bin/activate](#)).

```
pip3 install matplotlib scipy
```

4. Installing PyFlowCL/FV

Clone the appropriate repository:

```
# PyFlowCL  
git clone https://bitbucket.org/macart-group/pyflowcl
```



```
# PyFlowFV  
git clone https://bitbucket.org/macart-group/pyflowfv
```

If not already done, activate your PIP environment. We will now compile the Solver library for your system.

```
# (load your PIP environment and configure compilers)  
  
# PyFlowCL  
cd pyflowcl/src/PyFlowCL/Library  
python setup.py install  
  
# PyFlowFV  
cd pyflowfv/PyFlowFV/Library  
python setup.py install
```

If this finishes without error, then you are done and may run the code.

Congratulations!